

Javascript

```

        fnDBColumnInsert('LAB_NAME', fnTruncateToMax(sectionResults + '/entry['+i+']/organizer
/component['+j+']/observation/code/@displayName' + '|||' + sectionResults + '/entry['+i+']/org
anizer/component['+j+']/observation/code/translation/@displayName', 500));
        fnDBColumnInsert('LAB_CODE', fnTruncateToMax(sectionResults + '/entry['+i+']/organizer/
component['+j+']/observation/code/@code' + '|||' + sectionResults + '/entry['+i+']/organizer/com
ponent['+j+']/observation/code/translation/@code', 250));
        fnDBColumnInsert('LAB_CODE_SYSTEM', fnTruncateToMax(sectionResults + '/entry['+i+']/org
anizer/component['+j+']/observation/code/@codeSystem' + '|||' + sectionResults + '/entry['+i+']/
organizer/component['+j+']/observation/code/translation/@codeSystem', 250));
        fnDBColumnInsert('LAB_RESULT_NAME', fnTruncateToMax(sectionResults + '/entry['+i+']/org
anizer/component['+j+']/observation[templateId/@root="2.16.840.1.113883.10.20.22.4.2" or tem
plateId/@root="2.16.840.1.113883.10.20.15.2.3.2"]/value/@displayName', 500));
        var TRANSLATED_LAB_RESULT_NAME = qryConditionMedsAndLabs.getField('/message/LABORATORY
_TEST_CODES[LABORATORY_TEST_CODE="' + testCode + '" and CONDITION_CODE="' + relevantCondition +
'"]/LABORATORY_TEST_DESCRIPTION');
        if (TRANSLATED_LAB_RESULT_NAME != null) {
            fnDBColumnInsert('TRANSLATED_LAB_RESULT_NAME', TRANSLATED_LAB_RESULT_NAME.substring(0,
500));
        }
        fnDBColumnInsert('LAB_RESULT', fnTruncateToMax(sectionResults + '/entry['+i+']/organizer
/component['+j+']/observation[templateId/@root="2.16.840.1.113883.10.20.22.4.2" or templateId/
@root="2.16.840.1.113883.10.20.15.2.3.2"]/value' + '|||'
+ sectionResults + '/entry['+i+']/organizer/component['+j+']/observation[tem
plateId/@root="2.16.840.1.113883.10.20.22.4.2" or templateId/@root="2.16.840.1.113883.10.20.15
.2.3.2"]/value/originalText', 500));
        var LAB_RESULT_DATE = fnTruncateToMax(sectionResults + '/entry['+i+']/organizer/com
ponent['+j+']/observation[templateId/@root="2.16.840.1.113883.10.20.22.4.2" or templateId/@root
="2.16.840.1.113883.10.20.15.2.3.2"]/effectiveTime/@value' + '|||'
+ sectionResults + '/entry['+i+']/organizer/component['+j+']/observation[tem
plateId/@root="2.16.840.1.113883.10.20.22.4.2" or templateId/@root="2.16.840.1.113883.10.20.15
.2.3.2"]/effectiveTime/high/@value' + '|||'
+ sectionResults + '/entry['+i+']/organizer/component['+j+']/observation[tem
plateId/@root="2.16.840.1.113883.10.20.22.4.2" or templateId/@root="2.16.840.1.113883.10.20.15
.2.3.2"]/effectiveTime/low/@value', 100);
        fnDBColumnInsert('LAB_RESULT_DATE', LAB_RESULT_DATE);
        fnDBColumnInsert('LAB_RESULT_DATE_CONVERTED', fnConvertDateTimeField(LAB_RESULT_DATE));
        fnDBColumnInsert('LAB_INTERPRETATION', fnTruncateToMax(sectionResults + '/entry['+i+']/
organizer/component['+j+']/observation[templateId/@root="2.16.840.1.113883.10.20.22.4.2" or tem
plateId/@root="2.16.840.1.113883.10.20.15.2.3.2"]/interpretationCode/@code', 1500));
        fnDBColumnInsert('LAB_INTERPRETATION_VALUE', fnTruncateToMax(sectionResults + '/entry
['+i+']/organizer/component['+j+']/observation[templateId/@root="2.16.840.1.113883.10.20
.22.4.2" or templateId/@root="2.16.840.1.113883.10.20.15.2.3.2"]/interpretationCode/original
Text' + '|||'
+ sectionResults + '/entry['+i+']/organizer/component['+j+']/observation
[templateId/@root="2.16.840.1.113883.10.20.22.4.2" or templateId/@root="2.16.840.1.113883.10.20
.15.2.3.2"]/interpretationCode/@displayName', 2500));

        tempLAB_GROUP.appendChild(tempEle);
    } // if (fnIsRelevantLabTest(relevantCondition, testCode)) (END)
}
} // for (var i = 1; i < next.getRepeatCount(sectionResults + '/entry'); i++) (END)
} // if (next.getField(sectionResults + '/entry') != null) (END)

//----- PLAN OF TREATMENT section -----
//Loop through the PLAN OF TREATMENT section in the document and extract values only for the
lab tests relevant to the triggered condition.
if (next.getField(sectionPlanOfTreatment + '/entry') != null) {

```

Javascript

```

    for (var i = 1; i <= next.getRepeatCount(sectionPlanOfTreatment + '/entry'); i++) {
        var testCode = '';
        if (next.getField(sectionPlanOfTreatment + '/entry['+i+']/observation[templateId/@root="2.16.840.1.113883.10.20.22.4.44"]/code/@code') != null) {
            testCode = next.getField(sectionPlanOfTreatment + '/entry['+i+']/observation[templateId/@root="2.16.840.1.113883.10.20.22.4.44"]/code/@code');
        }
        else if (next.getField(sectionPlanOfTreatment + '/entry['+i+']/observation[templateId/@root="2.16.840.1.113883.10.20.22.4.44"]/code/translation/@code') != null) {
            testCode = next.getField(sectionPlanOfTreatment + '/entry['+i+']/observation[templateId/@root="2.16.840.1.113883.10.20.22.4.44"]/code/translation/@code');
        }

        if (fnIsRelevantLabTest(relevantCondition, testCode)) {
            tempEle = new XML(<row_eICR_DOCUMENT_LAB_ORDER></row_eICR_DOCUMENT_LAB_ORDER>);

            fnDBCcolumnInsert('REPORTABLE_CONDITION_ID', conditionCodeID);

            fnDBCcolumnInsert('LAB_ORDER_NAME', fnTruncateToMax(sectionPlanOfTreatment + '/entry['+i+']/observation[templateId/@root="2.16.840.1.113883.10.20.22.4.44"]/code/@displayName' + '||'
                + sectionPlanOfTreatment + '/entry['+i+']/observation[templateId/@root="2.16.840.1.113883.10.20.22.4.44"]/code/translation/@displayName', 500));
            fnDBCcolumnInsert('LAB_ORDER_CODE', testCode);
            var LAB_ORDER_DATE = fnTruncateToMax(sectionPlanOfTreatment + '/entry['+i+']/observation[templateId/@root="2.16.840.1.113883.10.20.22.4.44"]/effectiveTime/@value' + '||'
                + sectionPlanOfTreatment + '/entry['+i+']/observation[templateId/@root="2.16.840.1.113883.10.20.22.4.44"]/effectiveTime/high/@value' + '||'
                + sectionPlanOfTreatment + '/entry['+i+']/observation[templateId/@root="2.16.840.1.113883.10.20.22.4.44"]/effectiveTime/low/@value', 100);
            fnDBCcolumnInsert('LAB_ORDER_DATE', LAB_ORDER_DATE);
            fnDBCcolumnInsert('LAB_ORDER_DATE_CONVERTED', fnConvertDateTimeField(LAB_ORDER_DATE));

            tempLAB_ORDER_GROUP.appendChild(tempEle);
        } // if (fnIsRelevantLabTest(relevantCondition, testCode)) (END)
    } // for (var i = 1; i < next.getRepeatCount(sectionPlanOfTreatment + '/entry'); i++) (END)
} // if (next.getField(sectionPlanOfTreatment + '/entry') != null) (END)

} // if (conditionCodeID != null) (END)
} // for (var cntConditions = 0; cntConditions < arrConditions.length; cntConditions++) (END)

dbInsertsXML.appendChild(tempLAB_ORDER_GROUP);
dbInsertsXML.appendChild(tempLAB_GROUP);
} // if (next.getProperty('conditionCode') != null && next.getPropertyAsList('Condition_ID_Pairs') != null) (END)

//
=====
//_____ SET eICR_DOCUMENT_LAB_RESULT XML NODES (end
)
//
=====

//
=====

```

Javascript

```

//_____ SET eICR_DOCUMENT_TRIGGER XML NODES (start
)
//
=====
// Create XML nodes for key fields from the RR document to be written to the eICR_DOCUMENT
_TRIGGER table.
//If multiple conditions triggered the document, each property is delimited and will be split.

if (next.getProperty('conditionCode') != null && next.getPropertyAsList('Condition_ID_Pairs') !=
null) {
    var tempDOCUMENT_TRIGGER = new XML(<eICR_DOCUMENT_TRIGGER></eICR_DOCUMENT_TRIGGER>);

    //Most of the time, there will only be one condition but this while loop takes care of mul
tiple conditions.
    var arrConditions = next.getProperty('conditionCode').split('^');

    for (var cntConditions = 0; cntConditions < arrConditions.length; cntConditions++) {
        var relevantCondition = arrConditions[cntConditions];

        if (fnConditionCodeID(cntConditions) != null) {
            tempEle = new XML(<row_eICR_DOCUMENT_TRIGGER></row_eICR_DOCUMENT_TRIGGER>);

            fnDBColumnInsert('REPORTABLE_CONDITION_ID', fnConditionCodeID(cntConditions));
            fnDBColumnInsert('CODE_DETERMINATION', fnGetDocTriggerValue('rr_CODE_DETERMINATION', cntCondi
tions, 250));
            fnDBColumnInsert('RULE_DETERMINATION', fnGetDocTriggerValue('rr_RULE_DETERMINATION', cntCondi
tions, 1000));
            fnDBColumnInsert('TEXT_DETERMINATION', fnGetDocTriggerValue('rr_TEXT_DETERMINATION', cntCondi
tions, 1500));
            fnDBColumnInsert('LOCATION_RELEVANCE_CODE', fnGetDocTriggerValue('rr_LOCATION_RELEVANCE
_CODE', cntConditions, 500));
            fnDBColumnInsert('LOCATION_RELEVANCE_NAME', fnGetDocTriggerValue('rr_LOCATION_RELEVANCE
_NAME', cntConditions, 1500));
            fnDBColumnInsert('ROUTING_ENTITY_NAME', fnGetDocTriggerValue('rr_ROUTING_ENTITY_NAME', cnt
Conditions, 500));
            fnDBColumnInsert('ROUTING_ENTITY_DESCRIPTION', fnGetDocTriggerValue('rr_ROUTING_ENTITY_DES
CRIPTION', cntConditions, 5000));
            fnDBColumnInsert('RULES_AUTHORING_AGENCY', fnGetDocTriggerValue('rr_RULES_AUTHORING_AGENCY',
cntConditions, 500));
            fnDBColumnInsert('MANUALLY_TRIGGERED_FL', next.getProperty('rr_MANUALLY_TRIGGERED_FL'));

            tempDOCUMENT_TRIGGER.appendChild(tempEle);
        } // if (fnConditionCodeID(cntConditions) != null) (END)
    } // for (var cntConditions = 0; cntConditions < arrConditions.length; cntConditions++)
    (END)

    dbInsertsXML.appendChild(tempDOCUMENT_TRIGGER);
} // if (next.getProperty('conditionCode') != null && next.getPropertyAsList('Condition_ID
_Pairs') != null) (END)

//
=====
//_____ SET eICR_DOCUMENT_TRIGGER XML NODES (end
)
//
=====

next.text = dbInsertsXML;

```

Message Collector: Input Collation	
Collection Order	[ECLIR - eICR Message Parsing/8a93025a-63bf-4055-b1fb-51153a59a66e ,ECLIR - eICR Message Parsing/9911efcb-34a0-4e46-9c2a-5e365719d230]
Match Property	collectionID
Message Set Timeout	5
Timeout Units	Minutes
Timeout Destination	Error connector

Destinations

Name	Page
 db Insert eICR parsed data into ECLIR..... Database LookUp	143



db Insert eICR parsed data into ECLIR

Filter

Database LookUp

On ECLIR - eICR Message Parsing route

Notes

Purpose:

This Database Lookup filter:

Queries for an existing DOCUMENT_GROUP_ID based on the value in the SET_ID_CONCAT column in the eICR_DOCUMENT_GROUP table. If it doesn't exist, a new record is added and the generated DOCUMENT_GROUP_ID is used for inserts into the eICR_DOCUMENT_PARSED table to link all versions of an eICR document.

Iterates through the XML flat file created in the *js Parse eICR to XML for DB Inserts* JavaScript filter. The parsed eICR document data is written to the corresponding eICR tables in the ECLIR database.

Configuration Properties

Maximum Concurrency	1	
Database	[REDACTED]	
Host	[REDACTED]	
Port	[REDACTED]	
Database Name	[REDACTED]	
Username	[REDACTED]	
Password	[REDACTED]	
Additional Connection Properties	Property Name	Property Value
	[REDACTED]	[REDACTED]
Configuration File	[REDACTED]	
Message Type	XML	
Definition		
Query Timeout		

Configuration Properties

Deadlock Retry Attempts	5
Deadlock Retry Delay	100
Socket Timeout	0

Configuration File: Database LookUp.xml

```

<statement name="Query for existing DOCUMENT_GROUP_ID" ide:type="executeSql" ide:summary
="Check the eICR_DOCUMENT_GROUP table for existing, matching SET_ID_CONCAT for the document."
ide:allowCompoundStatements="true">
  <sql>SELECT TOP (1) DOCUMENT_GROUP_ID AS DocGroupID
FROM eICR_DOCUMENT_GROUP
WHERE SET_ID_CONCAT = `/message/eICR_DOCUMENT_PARSED/SET_ID_CONCAT`</sql>
  <substituteColumn name="DocGroupID">${DocGroupID}</substituteColumn>
  <resultSet>AllowNullResultSet</resultSet>
  <statement name="Create new DOCUMENT_GROUP_ID if it doesn't exist" ide:type="executeSql"
test="Null(@DocGroupID)" ide:summary="If a DOCUMENT_GROUP_ID was not found in the previous
query, create a new one." ide:allowCompoundStatements="true">
    <sql>INSERT INTO eICR_DOCUMENT_GROUP (
SET_ID_CONCAT
,INSERT_DATE
)
VALUES (
`/message/eICR_DOCUMENT_PARSED/SET_ID_CONCAT`
,SYSDATETIME()
)</sql>
    <statement ide:type="compoundSqlStatement">
      <sql>SELECT \@ \@IDENTITY AS DocGroupID</sql>
      <substituteColumn name="DocGroupID">${DocGroupID}</substituteColumn>
    </statement>
  </statement>
  <statement name="Insert Into eICR_DOCUMENT_PARSED Table" ide:type="executeSql" ide:summary
="Insert data into the eICR_DOCUMENT_PARSED table. One record per eICR message." ide:allowComp
oundStatements="true">
    <sql>INSERT INTO eICR_DOCUMENT_PARSED (
DOCUMENT_OID
,DOCUMENT_GROUP_ID
,DOC_ID_ROOT
,DOC_ID_EXTENSION
,SET_ID_CONCAT
,SET_ID_ROOT
,SET_ID_EXTENSION
,DOCUMENT_VERSION
,EHR_VENDOR_1
,EHR_VENDOR_SOFTWARE_VERSION_1
,EHR_VENDOR_2
,EHR_VENDOR_SOFTWARE_VERSION_2
,EHR_VENDOR_3
,EHR_VENDOR_SOFTWARE_VERSION_3
,DOCUMENT_CREATE_DATE
,DOCUMENT_CREATE_DATE_CONVERTED
,ENCOUNTER_DATE_LOW
,ENCOUNTER_DATE_LOW_CONVERTED

```

Configuration File: Database LookUp.xml

```

,ENCOUNTER_DATE_HIGH
,ENCOUNTER_DATE_HIGH_CONVERTED
,ENCOMPASSING_ENCOUNTER_CODE
,CUSTODIAN_ORG_ROOT_VALUE
,CUSTODIAN_ORG_EXTENSION_VALUE
,CUSTODIAN_ORG_NAME
,CUSTODIAN_ORG_ADDRESS_LINE
,CUSTODIAN_ORG_CITY
,CUSTODIAN_ORG_STATE
,CUSTODIAN_ORG_ZIP
,CUSTODIAN_ORG_COUNTY
,CUSTODIAN_ORG_PHONE
,CUSTODIAN_ORG_EMAIL
,SERVICE_PROVIDER_NAME
,SERVICE_PROVIDER_ADDRESS_LINE
,SERVICE_PROVIDER_CITY
,SERVICE_PROVIDER_STATE
,SERVICE_PROVIDER_ZIP
,SERVICE_PROVIDER_COUNTY
,HEALTHCARE_FACILITY_NAME
,HEALTHCARE_FACILITY_DISPLAY
,HEALTHCARE_FACILITY_ADDRESS_LINE
,HEALTHCARE_FACILITY_CITY
,HEALTHCARE_FACILITY_STATE
,HEALTHCARE_FACILITY_ZIP
,HEALTHCARE_FACILITY_COUNTY
,PROVIDER_FIRST_NAME
,PROVIDER_LAST_NAME
,PROVIDER_ADDRESS_LINE
,PROVIDER_CITY
,PROVIDER_STATE
,PROVIDER_ZIP
,REASON_FOR_VISIT
,ENCOUNTER_DETAILS_DESCRIPTION
,DISCHARGE_DISPOSITION_CODE
,DISCHARGE_DISPOSITION_TEXT
,PLANNED_ORDERS_FL
,DOCUMENT_SOURCE_BUCKET
,AIMS_SENDER
,AIMS_SENDER_NAME
,INSERT_DATE
)
VALUES (
~/message/eICR_DOCUMENT_PARSED/DOCUMENT_OID`
,@DocGroupID
~/message/eICR_DOCUMENT_PARSED/DOC_ID_ROOT`
~/message/eICR_DOCUMENT_PARSED/DOC_ID_EXTENSION`
~/message/eICR_DOCUMENT_PARSED/SET_ID_CONCAT`
~/message/eICR_DOCUMENT_PARSED/SET_ID_ROOT`
~/message/eICR_DOCUMENT_PARSED/SET_ID_EXTENSION`
~/message/eICR_DOCUMENT_PARSED/DOCUMENT_VERSION`
~/message/eICR_DOCUMENT_PARSED/EHR_VENDOR_1`
~/message/eICR_DOCUMENT_PARSED/EHR_VENDOR_SOFTWARE_VERSION_1`
~/message/eICR_DOCUMENT_PARSED/EHR_VENDOR_2`
~/message/eICR_DOCUMENT_PARSED/EHR_VENDOR_SOFTWARE_VERSION_2`
~/message/eICR_DOCUMENT_PARSED/EHR_VENDOR_3`
~/message/eICR_DOCUMENT_PARSED/EHR_VENDOR_SOFTWARE_VERSION_3`
~/message/eICR_DOCUMENT_PARSED/DOCUMENT_CREATE_DATE`

```

Configuration File: Database LookUp.xml

```

,~/message/eICR_DOCUMENT_PARSED/EVENT_DATE_CONVERTED`
,~/message/eICR_DOCUMENT_PARSED/ENCOUNTER_DATE_LOW`
,~/message/eICR_DOCUMENT_PARSED/ENCOUNTER_DATE_LOW_CONVERTED`
,~/message/eICR_DOCUMENT_PARSED/ENCOUNTER_DATE_HIGH`
,~/message/eICR_DOCUMENT_PARSED/ENCOUNTER_DATE_HIGH_CONVERTED`
,~/message/eICR_DOCUMENT_PARSED/ENCOMPASSING_ENCOUNTER_CODE`
,~/message/eICR_DOCUMENT_PARSED/CUSTODIAN_ORG_ROOT_VALUE`
,~/message/eICR_DOCUMENT_PARSED/CUSTODIAN_ORG_EXTENSION_VALUE`
,~/message/eICR_DOCUMENT_PARSED/CUSTODIAN_ORG_NAME`
,~/message/eICR_DOCUMENT_PARSED/CUSTODIAN_ORG_ADDRESS_LINE`
,~/message/eICR_DOCUMENT_PARSED/CUSTODIAN_ORG_CITY`
,~/message/eICR_DOCUMENT_PARSED/CUSTODIAN_ORG_STATE`
,~/message/eICR_DOCUMENT_PARSED/CUSTODIAN_ORG_ZIP`
,~/message/eICR_DOCUMENT_PARSED/CUSTODIAN_ORG_COUNTY`
,~/message/eICR_DOCUMENT_PARSED/CUSTODIAN_ORG_PHONE`
,~/message/eICR_DOCUMENT_PARSED/CUSTODIAN_ORG_EMAIL`
,~/message/eICR_DOCUMENT_PARSED/SERVICE_PROVIDER_NAME`
,~/message/eICR_DOCUMENT_PARSED/SERVICE_PROVIDER_ADDRESS_LINE`
,~/message/eICR_DOCUMENT_PARSED/SERVICE_PROVIDER_CITY`
,~/message/eICR_DOCUMENT_PARSED/SERVICE_PROVIDER_STATE`
,~/message/eICR_DOCUMENT_PARSED/SERVICE_PROVIDER_ZIP`
,~/message/eICR_DOCUMENT_PARSED/SERVICE_PROVIDER_COUNTY`
,~/message/eICR_DOCUMENT_PARSED/HEALTHCARE_FACILITY_NAME`
,~/message/eICR_DOCUMENT_PARSED/HEALTHCARE_FACILITY_DISPLAY`
,~/message/eICR_DOCUMENT_PARSED/HEALTHCARE_FACILITY_ADDRESS_LINE`
,~/message/eICR_DOCUMENT_PARSED/HEALTHCARE_FACILITY_CITY`
,~/message/eICR_DOCUMENT_PARSED/HEALTHCARE_FACILITY_STATE`
,~/message/eICR_DOCUMENT_PARSED/HEALTHCARE_FACILITY_ZIP`
,~/message/eICR_DOCUMENT_PARSED/HEALTHCARE_FACILITY_COUNTY`
,~/message/eICR_DOCUMENT_PARSED/PROVIDER_FIRST_NAME`
,~/message/eICR_DOCUMENT_PARSED/PROVIDER_LAST_NAME`
,~/message/eICR_DOCUMENT_PARSED/PROVIDER_ADDRESS_LINE`
,~/message/eICR_DOCUMENT_PARSED/PROVIDER_CITY`
,~/message/eICR_DOCUMENT_PARSED/PROVIDER_STATE`
,~/message/eICR_DOCUMENT_PARSED/PROVIDER_ZIP`
,~/message/eICR_DOCUMENT_PARSED/REASON_FOR_VISIT`
,~/message/eICR_DOCUMENT_PARSED/ENCOUNTER_DETAILS_DESCRIPTION`
,~/message/eICR_DOCUMENT_PARSED/DISCHARGE_DISPOSITION_CODE`
,~/message/eICR_DOCUMENT_PARSED/DISCHARGE_DISPOSITION_TEXT`
,~/message/eICR_DOCUMENT_PARSED/PLANNED_ORDERS_FL`
,$DOCUMENT_SOURCE_BUCKET
,$saimsSender
,$saimsSenderName
,$SYSDATETIME()
)</sql>
<statement ide:type="compoundSqlStatement">
  <sql>SELECT \@ \@IDENTITY AS eICR_DOCUMENT_PARSED_ID</sql>
  <substituteColumn name="eICR_DOCUMENT_PARSED_ID">$eICR_DOCUMENT_PARSED_ID</substitute
Column>
</statement>
<statement name="Insert Into eICR_DOCUMENT_PATIENT Table" ide:type="executeSql" ide:summary
="Insert data into the eICR_DOCUMENT_PATIENT table. One record per eICR message." ide:allow
CompoundStatements="true">
  <sql>INSERT INTO eICR_DOCUMENT_PATIENT (
eICR_DOCUMENT_PARSED_ID
,MRN
,PATIENT_FIRST_NAME
,PATIENT_MIDDLE_NAME

```

Configuration File: Database LookUp.xml

```

,PATIENT_LAST_NAME
,PATIENT_SUFFIX
,PATIENT_ADDRESS_LINE
,PATIENT_CITY
,PATIENT_STATE
,PATIENT_ZIP
,PATIENT_COUNTY
,PATIENT_COUNTRY
,PATIENT_PHONE
,PATIENT_PHONE_FORMATTED
,PATIENT_EMAIL
,PATIENT_BIRTH_SEX
,PATIENT_CURRENT_SEX
,PATIENT_LANGUAGE
,PATIENT_DOB
,PATIENT_DOB_CONVERTED
,PATIENT_RACE
,PATIENT_RACE_CODE_SYSTEM
,PATIENT_RACE_NAME
,PATIENT_ETHNICITY
,PATIENT_ETHNICITY_CODE_SYSTEM
,PATIENT_ETHNICITY_NAME
,PATIENT_SSN_FL
,PATIENT_MARITAL_STATUS
,PATIENT_MARITAL_STATUS_CODE_SYSTEM
,PATIENT_MARITAL_STATUS_NAME
,PATIENT_DECEASED_INDICATOR
,PATIENT_DECEASED_DATE
,PATIENT_DECEASED_DATE_CONVERTED
,PATIENT_PREGNANCY_STATUS
,PATIENT_INFERRED_PREGNANCY_STATUS
,PATIENT_PREGNANCY_NOTES_FL
,PATIENT_TRAVEL_HISTORY_LIKELY_FL
,PATIENT_SMOKING_CODE
,PATIENT_SMOKING_CODE_SYSTEM
,PATIENT_SMOKING_CODE_DISPLAY
,PATIENT_ALCOHOL_CODE
,PATIENT_ALCOHOL_CODE_SYSTEM
,PATIENT_ALCOHOL_CODE_DISPLAY
,PATIENT_TOBACCO_CODE
,PATIENT_TOBACCO_CODE_SYSTEM
,PATIENT_TOBACCO_CODE_DISPLAY
,PATIENT_HPI_FL
,PATIENT_IMMUNIZATION_HISTORY_FL
,PATIENT_INSERT_DATE
)
VALUES (
@eICR_DOCUMENT_PARSED_ID
,~/message/eICR_DOCUMENT_PATIENT/MRN~
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_FIRST_NAME~
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_MIDDLE_NAME~
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_LAST_NAME~
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_SUFFIX~
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_ADDRESS_LINE~
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_CITY~
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_STATE~
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_ZIP~
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_COUNTY~

```

Configuration File: Database LookUp.xml

```

,~/message/eICR_DOCUMENT_PATIENT/PATIENT_COUNTRY`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_PHONE`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_PHONE_FORMATTED`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_EMAIL`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_BIRTH_SEX`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_CURRENT_SEX`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_LANGUAGE`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_DOB`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_DOB_CONVERTED`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_RACE`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_RACE_CODE_SYSTEM`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_RACE_NAME`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_ETHNICITY`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_ETHNICITY_CODE_SYSTEM`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_ETHNICITY_NAME`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_SSN_FL`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_MARITAL_STATUS`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_MARITAL_STATUS_CODE_SYSTEM`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_MARITAL_STATUS_NAME`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_DECEASED_INDICATOR`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_DECEASED_DATE`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_DECEASED_DATE_CONVERTED`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_PREGNANCY_STATUS`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_INFERRED_PREGNANCY_STATUS`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_PREGNANCY_NOTES_FL`
,~/message/eICR_DOCUMENT_PATIENT/TRAVEL_HISTORY_LIKELY_FL`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_SMOKING_CODE`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_SMOKING_CODE_SYSTEM`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_SMOKING_CODE_DISPLAY`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_ALCOHOL_CODE`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_ALCOHOL_CODE_SYSTEM`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_ALCOHOL_CODE_DISPLAY`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_TOBACCO_CODE`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_TOBACCO_CODE_SYSTEM`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_TOBACCO_CODE_DISPLAY`
,~/message/eICR_DOCUMENT_PATIENT/PATIENT_HPI_FL`
,~/message/eICR_DOCUMENT_PATIENT/IMMUNIZATION_HISTORY_FL`
,SYSDATETIME()
)</sql>
</statement>
<statement name="Insert Into eICR_DOCUMENT_MEDICATION Table" ide:type="executeSql" iterate
="eICR_DOCUMENT_MEDICATION/row_eICR_DOCUMENT_MEDICATION" ide:summary="Insert data into the eICR
DOCUMENT_MEDICATION table. One record per medication relevant to the reportable condition."
ide:allowCompoundStatements="true" user:iterationRootMessagePath="message">
<sql>INSERT INTO eICR_DOCUMENT_MEDICATION (
eICR_DOCUMENT_PARSED_ID
,REPORTABLE_CONDITION_ID
,MEDICATION_CODE
,MEDICATION_NAME
,MEDICATION_DOSE_VALUE
,MEDICATION_DOSE_UNIT
,MEDICATION_START_TIME
,MEDICATION_START_TIME_CONVERTED
,MEDICATION_END_TIME
,MEDICATION_END_TIME_CONVERTED
,MEDICATION_STATUS_CODE
,MEDICATION_ORDERED_FL
,MEDICATION_ABORTED_FL

```

Configuration File: Database LookUp.xml

```

,MEDICATION_COMPLETED_FL
,MEDICATION_ROUTE_GIVEN
,MEDICATION_ROUTE_ORDERED
,INSERT_DATE
)
VALUES (
  @eICR_DOCUMENT_PARSED_ID
  ,~/message/eICR_DOCUMENT_MEDICATION/row_eICR_DOCUMENT_MEDICATION/REPORTABLE_CONDITION_ID`
  ,~/message/eICR_DOCUMENT_MEDICATION/row_eICR_DOCUMENT_MEDICATION/MEDICATION_CODE`
  ,~/message/eICR_DOCUMENT_MEDICATION/row_eICR_DOCUMENT_MEDICATION/MEDICATION_NAME`
  ,~/message/eICR_DOCUMENT_MEDICATION/row_eICR_DOCUMENT_MEDICATION/MEDICATION_DOSE_VALUE`
  ,~/message/eICR_DOCUMENT_MEDICATION/row_eICR_DOCUMENT_MEDICATION/MEDICATION_DOSE_UNIT`
  ,~/message/eICR_DOCUMENT_MEDICATION/row_eICR_DOCUMENT_MEDICATION/MEDICATION_START_TIME`
  ,~/message/eICR_DOCUMENT_MEDICATION/row_eICR_DOCUMENT_MEDICATION/MEDICATION_START_TIME_CON
VERTED`
  ,~/message/eICR_DOCUMENT_MEDICATION/row_eICR_DOCUMENT_MEDICATION/MEDICATION_END_TIME`
  ,~/message/eICR_DOCUMENT_MEDICATION/row_eICR_DOCUMENT_MEDICATION/MEDICATION_END_TIME_CON
VERTED`
  ,~/message/eICR_DOCUMENT_MEDICATION/row_eICR_DOCUMENT_MEDICATION/MEDICATION_STATUS_CODE`
  ,~/message/eICR_DOCUMENT_MEDICATION/row_eICR_DOCUMENT_MEDICATION/MEDICATION_ORDERED_FL`
  ,~/message/eICR_DOCUMENT_MEDICATION/row_eICR_DOCUMENT_MEDICATION/MEDICATION_ABORTED_FL`
  ,~/message/eICR_DOCUMENT_MEDICATION/row_eICR_DOCUMENT_MEDICATION/MEDICATION_COMPLETED_FL`
  ,~/message/eICR_DOCUMENT_MEDICATION/row_eICR_DOCUMENT_MEDICATION/MEDICATION_ROUTE_GIVEN`
  ,~/message/eICR_DOCUMENT_MEDICATION/row_eICR_DOCUMENT_MEDICATION/MEDICATION_ROUTE_ORDERED`
  ,SYSDATETIME()
)
</sql>
</statement>
<statement name="Insert Into eICR_DOCUMENT_LAB_ORDER Table" ide:type="executeSql" iterate
="eICR_DOCUMENT_LAB_ORDER/row_eICR_DOCUMENT_LAB_ORDER" ide:summary="Insert data into the eICR
_DOCUMENT_LAB_ORDER table. One record per lab order relevant to the reportable condition." ide
:allowCompoundStatements="true">
  <sql>INSERT INTO eICR_DOCUMENT_LAB_ORDER (
    eICR_DOCUMENT_PARSED_ID
    ,REPORTABLE_CONDITION_ID
    ,LAB_ORDER_NAME
    ,LAB_ORDER_CODE
    ,LAB_ORDER_DATE
    ,LAB_ORDER_DATE_CONVERTED
    ,INSERT_DATE
  )
  VALUES (
    @eICR_DOCUMENT_PARSED_ID
    ,~/message/eICR_DOCUMENT_LAB_ORDER/row_eICR_DOCUMENT_LAB_ORDER/REPORTABLE_CONDITION_ID`
    ,~/message/eICR_DOCUMENT_LAB_ORDER/row_eICR_DOCUMENT_LAB_ORDER/LAB_ORDER_NAME`
    ,~/message/eICR_DOCUMENT_LAB_ORDER/row_eICR_DOCUMENT_LAB_ORDER/LAB_ORDER_CODE`
    ,~/message/eICR_DOCUMENT_LAB_ORDER/row_eICR_DOCUMENT_LAB_ORDER/LAB_ORDER_DATE`
    ,~/message/eICR_DOCUMENT_LAB_ORDER/row_eICR_DOCUMENT_LAB_ORDER/LAB_ORDER_DATE_CONVERTED`
    ,SYSDATETIME()
  )
</sql>
</statement>
<statement name="Insert Into eICR_DOCUMENT_LAB_RESULT Table" ide:type="executeSql" iterate
="eICR_DOCUMENT_LAB_RESULT/row_eICR_DOCUMENT_LAB_RESULT" ide:summary="Insert data into the eICR
_DOCUMENT_LAB_RESULT table. One record per lab result relevant to the reportable condition."
ide:allowCompoundStatements="true">
  <sql>INSERT INTO eICR_DOCUMENT_LAB_RESULT (
    eICR_DOCUMENT_PARSED_ID
    ,REPORTABLE_CONDITION_ID
    ,LAB_NAME

```

Configuration File: Database LookUp.xml

```

,LAB_CODE
,LAB_CODE_SYSTEM
,LAB_RESULT_NAME
,TRANSLATED_LAB_RESULT_NAME
,LAB_RESULT
,LAB_RESULT_DATE
,LAB_RESULT_DATE_CONVERTED
,LAB_INTERPRETATION
,LAB_INTERPRETATION_VALUE
,INSERT_DATE
)
VALUES (
  @eICR_DOCUMENT_PARSED_ID
  ,~/message/eICR_DOCUMENT_LAB_RESULT/row_eICR_DOCUMENT_LAB_RESULT/REPORTABLE_CONDITION_ID`
  ,~/message/eICR_DOCUMENT_LAB_RESULT/row_eICR_DOCUMENT_LAB_RESULT/LAB_NAME`
  ,~/message/eICR_DOCUMENT_LAB_RESULT/row_eICR_DOCUMENT_LAB_RESULT/LAB_CODE`
  ,~/message/eICR_DOCUMENT_LAB_RESULT/row_eICR_DOCUMENT_LAB_RESULT/LAB_CODE_SYSTEM`
  ,~/message/eICR_DOCUMENT_LAB_RESULT/row_eICR_DOCUMENT_LAB_RESULT/LAB_RESULT_NAME`
  ,~/message/eICR_DOCUMENT_LAB_RESULT/row_eICR_DOCUMENT_LAB_RESULT/TRANSLATED_LAB_RESULT_NAME`
  ,~/message/eICR_DOCUMENT_LAB_RESULT/row_eICR_DOCUMENT_LAB_RESULT/LAB_RESULT`
  ,~/message/eICR_DOCUMENT_LAB_RESULT/row_eICR_DOCUMENT_LAB_RESULT/LAB_RESULT_DATE`
  ,~/message/eICR_DOCUMENT_LAB_RESULT/row_eICR_DOCUMENT_LAB_RESULT/LAB_RESULT_DATE_CONVERTED`
  ,~/message/eICR_DOCUMENT_LAB_RESULT/row_eICR_DOCUMENT_LAB_RESULT/LAB_INTERPRETATION`
  ,~/message/eICR_DOCUMENT_LAB_RESULT/row_eICR_DOCUMENT_LAB_RESULT/LAB_INTERPRETATION_VALUE`
  ,SYSDATETIME()
)
</sql>
</statement>
<statement name="Insert Into eICR_REPORTABLE_RESPONSE Table" ide:type="executeSql" ide:sum
mary="Insert data into the eICR_REPORTABLE_RESPONSE table. One record per RR message." ide
:allowCompoundStatements="true">
  <sql>INSERT INTO eICR_REPORTABLE_RESPONSE (
    eICR_DOCUMENT_PARSED_ID
    ,ORIGINAL_FILE_NAME
    ,INITIATION_INDICATOR
    ,GLOBAL_UNIQUE_ID
    ,eRSD_VERSION_20
    ,eRSD_VERSION_29
    ,eRSD_VERSION_31
    ,INSERT_DATE
  )
  VALUES (
    @eICR_DOCUMENT_PARSED_ID
    ,$rr_ORIGINAL_FILE_NAME
    , $rr_INITIATION_INDICATOR
    , $rr_GLOBAL_UNIQUE_ID
    , $rr_eRSD_VERSION_20
    , $rr_eRSD_VERSION_29
    , $rr_eRSD_VERSION_31
    ,SYSDATETIME()
  )
</sql>
</statement>
<statement name="Insert Into eICR_DOCUMENT_TRIGGER Table" ide:type="executeSql" iterate
="eICR_DOCUMENT_TRIGGER/row_eICR_DOCUMENT_TRIGGER" ide:summary="Insert data into the eICR_DOC
UMENT_TRIGGER table. One record per reportable condition in the RR message." ide:allowCompound
Statements="true">
  <sql>INSERT INTO eICR_DOCUMENT_TRIGGER (
    REPORTABLE_CONDITION_ID
    ,eICR_DOCUMENT_PARSED_ID

```

Configuration File: Database LookUp.xml

```

, CODE_DETERMINATION
, RULE_DETERMINATION
, TEXT_DETERMINATION
, LOCATION_RELEVANCE_CODE
, LOCATION_RELEVANCE_NAME
, ROUTING_ENTITY_NAME
, ROUTING_ENTITY_DESCRIPTION
, RULES_AUTHORIZING_AGENCY
, MANUALLY_TRIGGERED_FL
, INSERT_DATE
)
VALUES (
  `/message/eICR_DOCUMENT_TRIGGER/row_eICR_DOCUMENT_TRIGGER/REPORTABLE_CONDITION_ID`
, @eICR_DOCUMENT_PARSED_ID
, `/message/eICR_DOCUMENT_TRIGGER/row_eICR_DOCUMENT_TRIGGER/CODE_DETERMINATION`
, `/message/eICR_DOCUMENT_TRIGGER/row_eICR_DOCUMENT_TRIGGER/RULE_DETERMINATION`
, `/message/eICR_DOCUMENT_TRIGGER/row_eICR_DOCUMENT_TRIGGER/TEXT_DETERMINATION`
, `/message/eICR_DOCUMENT_TRIGGER/row_eICR_DOCUMENT_TRIGGER/LOCATION_RELEVANCE_CODE`
, `/message/eICR_DOCUMENT_TRIGGER/row_eICR_DOCUMENT_TRIGGER/LOCATION_RELEVANCE_NAME`
, `/message/eICR_DOCUMENT_TRIGGER/row_eICR_DOCUMENT_TRIGGER/ROUTING_ENTITY_NAME`
, `/message/eICR_DOCUMENT_TRIGGER/row_eICR_DOCUMENT_TRIGGER/ROUTING_ENTITY_DESCRIPTION`
, `/message/eICR_DOCUMENT_TRIGGER/row_eICR_DOCUMENT_TRIGGER/RULES_AUTHORIZING_AGENCY`
, `/message/eICR_DOCUMENT_TRIGGER/row_eICR_DOCUMENT_TRIGGER/MANUALLY_TRIGGERED_FL`
, SYSDATETIME()
) </sql>
</statement>
</statement>
</config>

```

Destinations

Name	Page
 no-op to Sink..... No-operation	155

**no-op REH to Sink****No-operation**

On ECLIR - eICR Message Parsing route

[Filter](#)**Notes**

Purpose: This No-operation filter sends messages from the *ECLIR - eICR Message Parsing Route Error Handler* to the *ECLIR - eICR Route Error Handler Sink* after the errors have been stored into the eICR_ERROR table in the ECLIR database.

Destinations

Name	Page
 ECLIR - eICR Route Error Handler Sink..... Sink	11



no-op eICR Msgs In

[Filter](#)

No-operation

On ECLIR - eICR Message Parsing route

Notes

Purpose: This No-operation filter receives eICR messages from the *ECLIR - eICR Condition Checks* route.

Destinations

Name	Page
js Parse eICR to XML for DB Inserts..... JavaScript (v.1)	119



no-op Relevant Meds and Labs In

[Filter](#)

No-operation

On ECLIR - eICR Message Parsing route

Notes

Purpose: This No-operation filter receives an XML file of medications and lab tests relevant to the condition(s) from the *ECLIR - eICR Condition Checks* route.

Destinations

Name	Page
js Parse eICR to XML for DB Inserts..... JavaScript (v.1)	119



no-op Route Error Handler In

[Filter](#)

No-operation

On ECLIR - eICR Message Parsing route

Notes

Purpose: This No-operation filter receives messages from the *ECLIR - eICR Message Parsing Route Error Handler* when an unhandled error occurs anywhere in the route.

Destinations

Name	Page
 js Set Error properties.....	JavaScript (v.1) 153



js Set Error properties

JavaScript (v.1)

On ECLIR - eICR Message Parsing route

Filter

Notes

Purpose: This JavaScript filter

Sets the property ERROR_CATEGORY with the type of error to be stored in the ERROR_CATEGORY column in the eICR_ERROR table. The category is set to 'FATAL' because the message stops processing.

Sets the property ERROR_TYPE with the type of error to be stored in the ERROR_TYPE column in the eICR_ERROR table. The type is set to 'Rhapsody' because the error is an internal Rhapsody error.

Sets the property PROCESSING_STEP with the Rhapsody route name and filter name to be stored in the PROCESSING_STEP column in the eICR_ERROR table.

Sets the property ERROR_MSG with the Rhapsody generated errors to be stored in the ERROR_MSG column in the eICR_ERROR table.

Sets the property DOCUMENT_SOURCE_BUCKET based on the rhapsody:InputCommunicationPoint message property from the  legacy routes to be stored in the ERROR_MSG column in the eICR_ERROR table.

Javascript

```
//
=====
// _____ INITIAL VARIABLES (start
)
//
=====

var next = output.append(input[0]);

//
=====
// _____ INITIAL VARIABLES (end
)
//
=====

//
=====
// _____ SET MESSAGE PROPERTIES FOR DB INSERT (start
)
//
=====
```

Javascript

```

next.setProperty('ERROR_CATEGORY', 'FATAL');
next.setProperty('ERROR_TYPE', 'Rhapsody');
next.setProperty('PROCESSING_STEP', 'Parsing: ECLIR - eICR Message Parsing (Route) ->' + next
.getProperty('rhapsody:ErrorFilterName') + ' (Filter)');
//next.setProperty('ERROR_MSG', 'Internal Rhapsody Route Error. Message processing could not
continue: ' + next.getErrors());
var ERROR_MSG = 'Internal Rhapsody Route Error. Message processing could not continue: ' + next
.getErrors();
next.setProperty('ERROR_MSG', ERROR_MSG);
//Send the message to the Error Queue if it was caused by a database connectivity issue.
if (ERROR_MSG.indexOf('The connection is broken and recovery is not possible.') != -1 || ERROR
_MSG == 'Internal Rhapsody Route Error. Message processing could not continue: ') {
    next.setProperty('SendToErrorQueue', true);
}

//Set property for 'DOCUMENT_SOURCE_BUCKET' column based on the input comm point.
var DOCUMENT_SOURCE_BUCKET = next.getProperty('rhapsody:InputCommunicationPoint');
if (DOCUMENT_SOURCE_BUCKET == 'AIMS Platform') {
    DOCUMENT_SOURCE_BUCKET = '██████_PROD';
}
else {
    DOCUMENT_SOURCE_BUCKET = '██████_TEST';
}
next.setProperty('DOCUMENT_SOURCE_BUCKET', DOCUMENT_SOURCE_BUCKET);

//
=====
//_____ SET MESSAGE PROPERTIES FOR DB INSERT (end
)
//
=====

```

Destinations

Name	Page
Output Condition	
SendToErrorQueue = 'true' ignoring case	
! Error Queue..... Error Queue	154
Error Connector	Page
db eICR Message Parsing Route Errors..... Database LookUp	117

**Error Queue**

Filter

Error Queue

On ECLIR - eICR Message Parsing route

Configuration Properties

Error Message	Sent from error filter
---------------	------------------------



no-op to Sink

No-operation

On ECLIR - eICR Message Parsing route

Filter

Notes

Purpose: This No-operation filter sends all messages to the *ECLIR - eCR Sink* after queries are completed.

Destinations

Name	Page
ECLIR - eCR Sink.....Sink	12



ELR

Folder

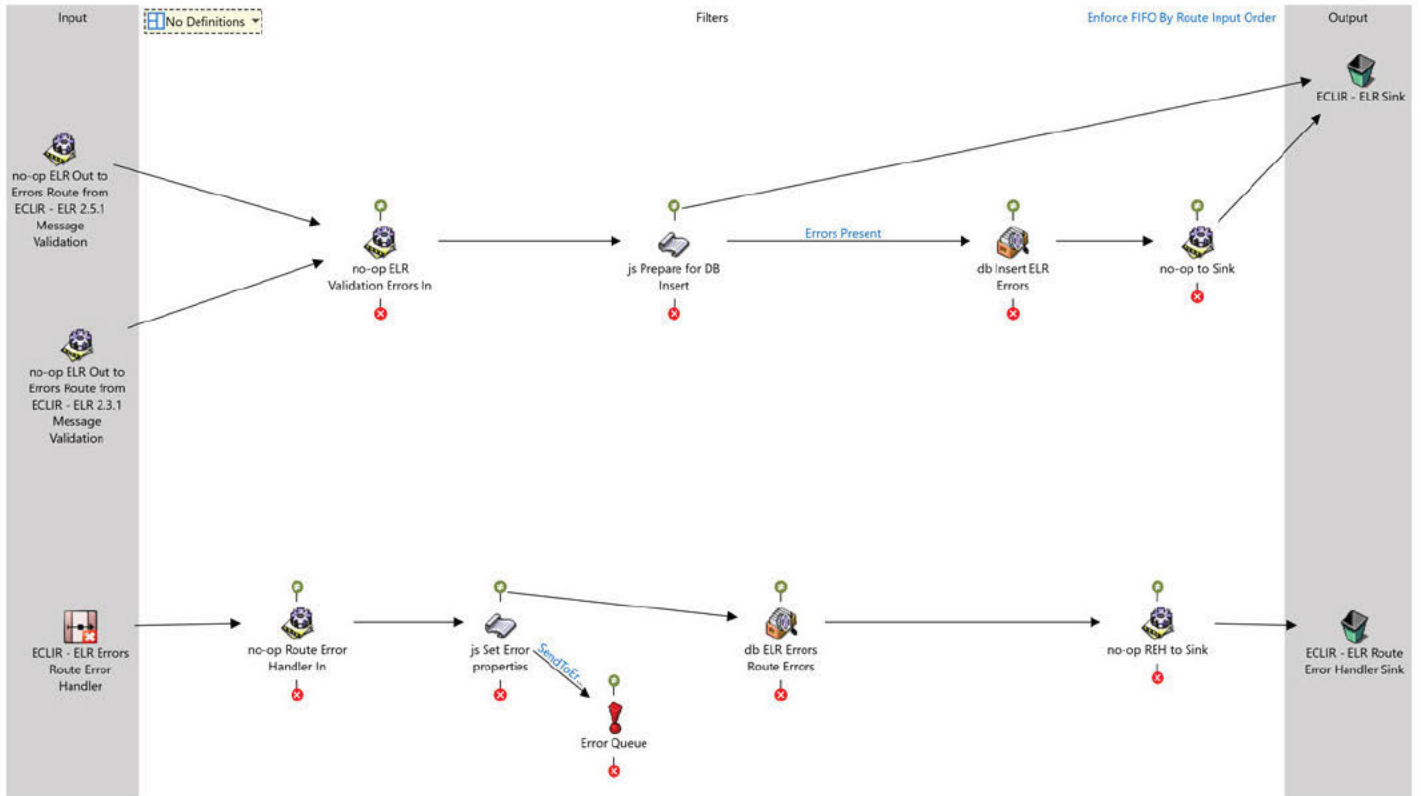
Full Path /ECLIR/Route__ECLIR/ELR

Summary		
Routes (3)		
Sub-folders (3)		
Folder Contents		Page
ECLIR - ELR Errors.....	Route	157
ECLIR - ELR Legacy Route Errors.....	Route	169
ECLIR - ELR non-NBS Subscriptions.....	Route	185
ECLIR - ELR Parsing.....	Folder	194
ECLIR - ELR Polling.....	Folder	302
ECLIR - ELR Validation.....	Folder	352



ECLIR - ELR Errors

Route



Notes

Purpose: This route has 2 main functions:

Receive ELR validation errors from the *ECLIR - ELR 2.3.1 Message Parsing* and *ECLIR - ELR 2.3.1 Message Parsing* routes to be recorded in the ELR_ERROR table in ECLIR.

Receive errors from [REDACTED] legacy routes to be recorded in the ELR_ERROR table in ECLIR.

Inputs

		Page
no-op ELR Out to Errors Route (filter from "ECLIR - ELR 2.5.1 Message Validation").....	No-operation	410
no-op ELR Out to Errors Route (filter from "ECLIR - ELR 2.3.1 Message Validation").....	No-operation	356

Outputs

		Page
ECLIR - ELR Sink.....	Sink	14
ECLIR - ELR Route Error Handler Sink.....	Sink	13

Filters

Name		Page
db ELR Errors Route Errors.....	Database LookUp	166
db Insert ELR Errors.....	Database LookUp	159

Name	Page
 Error Queue..... Error Queue	168
 js Prepare for DB Insert..... JavaScript (v.1)	164
 js Set Error properties..... JavaScript (v.1)	162
 no-op ELR Validation Errors In..... No-operation	159
 no-op REH to Sink..... No-operation	162
 no-op Route Error Handler In..... No-operation	162
 no-op to Sink..... No-operation	165

Details

General	
Priority	Normal
Startup State	Startup Third
FIFO Mode	Enforce FIFO By Route Input Order
Stop route on error message	False



no-op ELR Validation Errors In

Filter

No-operation

On ECLIR - ELR Errors route

Notes

Purpose: This No-operation filter receives XML messages created to record any ELR message validation errors to be recorded in the ECLIR database

Destinations

Name	Page
js Prepare for DB Insert.....JavaScript (v.1)	164



db Insert ELR Errors

Filter

Database LookUp

On ECLIR - ELR Errors route

Notes

Purpose: This Database Lookup filter inserts an error record into the ELR_ERROR table in the ECLIR database by looping through an XML representation of the error record created in the previous route. This filter will also update the SUCCESS_FL column in the ELR_MESSAGE table if a FATAL error was recorded.

Configuration Properties

Maximum Concurrency	1	
Database		
Host		
Port		
Database Name		
Username		
Password		
Additional Connection Properties	Property Name	Property Value
Configuration File		
Message Type	XML	
Definition		
Query Timeout		
Deadlock Retry Attempts	5	

Configuration Properties

Deadlock Retry Delay	100
Socket Timeout	0

Configuration File: Database LookUp 2.xml

```

<statement name="ELR_MESSAGE ID Does Not exist" ide:type="executeSql" test="Null($ELR_MESSAGE_ID)" ide:summary="The ELR_MESSAGE ID does not exist in the ELR_MESSAGE table. Create an entry in that table to create an ID." ide:allowCompoundStatements="true">
  <sql>INSERT INTO ELR_MESSAGE (
ORIGINAL_MESSAGE
, █████ ENVIRONMENT
,SUCCESS_FL
,INSERT_DATE
)
VALUES (
$messageContent
,$█████ ENVIRONMENT
,0
,SYSDATETIME()
)</sql>
  <statement ide:type="compoundSqlStatement">
    <sql>SELECT \@ \@IDENTITY AS ELR_MESSAGE_ID</sql>
    <substituteColumn name="ELR_MESSAGE_ID">$ELR_MESSAGE_ID</substituteColumn>
  </statement>
  <statement name="Insert into ELR_ERROR table" ide:type="executeSql" iterate="errorRecord" test="NotNull(@ELR_MESSAGE_ID)" ide:summary="Add error record to the ELR_ERROR table." ide:allowCompoundStatements="true" user:iterationRootMessagePath="message">
    <sql>INSERT INTO ELR_ERROR (
ELR_MESSAGE_ID
,VALIDATION_TYPE
,ERROR_CATEGORY
,ERROR_TYPE
,ERROR_TYPE_DETAIL
,PROCESSING_STEP
,FIELD
,FIELD_DESCRIPTION
,FIELD_PATH
,FIELD_VALUE
,ERROR_MSG
,INSERT_DATE
)
VALUES (
@ELR_MESSAGE_ID
,~/message/errorRecord/VALIDATION_TYPE~
,~/message/errorRecord/ERROR_CATEGORY~
,~/message/errorRecord/ERROR_TYPE~
,~/message/errorRecord/ERROR_TYPE_DETAIL~
,~/message/errorRecord/PROCESSING_STEP~
,~/message/errorRecord/FIELD~
,~/message/errorRecord/FIELD_DESCRIPTION~
,~/message/errorRecord/FIELD_PATH~
,~/message/errorRecord/FIELD_VALUE~
,~/message/errorRecord/ERROR_MSG~
,SYSDATETIME()
)</sql>

```

Configuration File: Database LookUp 2.xml

```

    <statement name="Update SUCCESS_FL" ide:type="executeSql" test="Equals(`/message/error
Record/ERROR_CATEGORY`, 'FATAL')\" ide:summary="Updates the SUCCESS_FL column in the ELR_MESSAGE
table if a FATAL error was recorded." ide:allowCompoundStatements="true">
        <sql>UPDATE ELR_MESSAGE SET
        SUCCESS_FL = 0
        WHERE ID = @ELR_MESSAGE_ID</sql>
    </statement>
</statement>
</statement>
    <statement name="Insert into ELR_ERROR table" ide:type="executeSql" iterate="errorRecord"
test="NotNull($ELR_MESSAGE_ID)" ide:summary="Add error record to the ELR_ERROR table." ide
:allowCompoundStatements="true" user:iterationRootMessagePath="message">
        <sql>INSERT INTO ELR_ERROR (
        ELR_MESSAGE_ID
        ,VALIDATION_TYPE
        ,ERROR_CATEGORY
        ,ERROR_TYPE
        ,ERROR_TYPE_DETAIL
        ,PROCESSING_STEP
        ,FIELD
        ,FIELD_DESCRIPTION
        ,FIELD_PATH
        ,FIELD_VALUE
        ,ERROR_MSG
        ,INSERT_DATE
        )
        VALUES (
        `/message/errorRecord/ELR_MESSAGE_ID`
        , `/message/errorRecord/VALIDATION_TYPE`
        , `/message/errorRecord/ERROR_CATEGORY`
        , `/message/errorRecord/ERROR_TYPE`
        , `/message/errorRecord/ERROR_TYPE_DETAIL`
        , `/message/errorRecord/PROCESSING_STEP`
        , `/message/errorRecord/FIELD`
        , `/message/errorRecord/FIELD_DESCRIPTION`
        , `/message/errorRecord/FIELD_PATH`
        , `/message/errorRecord/FIELD_VALUE`
        , `/message/errorRecord/ERROR_MSG`
        ,SYSDATETIME()
        )</sql>
    <statement name="Update SUCCESS_FL" ide:type="executeSql" test="Equals(`/message/error
Record/ERROR_CATEGORY`, 'FATAL')\" ide:summary="Updates the SUCCESS_FL column in the ELR_MESSAGE
table if a FATAL error was recorded." ide:allowCompoundStatements="true">
        <sql>UPDATE ELR_MESSAGE SET
        SUCCESS_FL = 0
        WHERE ID = `/message/errorRecord/ELR_MESSAGE_ID`</sql>
    </statement>
</statement>
</config>

```

Destinations

Name	Page
 no-op to Sink..... No-operation	165



no-op Route Error Handler In

Filter

No-operation

On ECLIR - ELR Errors route

Notes

Purpose: This No-operation filter receives messages from the *ECLIR - ELR Errors Route Error Handler* when an unhandled error occurs anywhere in the route.

Destinations

Name	Page
js Set Error properties..... JavaScript (v.1)	162



no-op REH to Sink

Filter

No-operation

On ECLIR - ELR Errors route

Notes

Purpose: This No-operation filter sends messages from the *ECLIR - ELR Errors Route Error Handler* to the *ECLIR - ELR Route Error Handler Sink* after the errors have been stored into the ELR_ERROR table in the ECLIR database.

Destinations

Name	Page
ECLIR - ELR Route Error Handler Sink..... Sink	13



js Set Error properties

Filter

JavaScript (v.1)

On ECLIR - ELR Errors route

Notes

Purpose: This JavaScript filter

Sets the property VALIDATION_TYPE with the type of validation error to be stored in the VALIDATION_TYPE column in the ELR_ERROR table. The type is set to 'Rhapsody' because the error is an internal Rhapsody error.

Sets the property ERROR_CATEGORY with the category of the error to be stored in the ERROR_CATEGORY column in the ELR_ERROR table. The category is set to 'FATAL' because the message stops processing.

Notes

Sets the property ERROR_TYPE with the type of error to be stored in the ERROR_TYPE column in the ELR_ERROR table. The type is set to 'Rhapsody' because the error is an internal Rhapsody error.

Sets the property PROCESSING_STEP with the Rhapsody route name and filter name to be stored in the PROCESSING_STEP column in the ELR_ERROR table.

Sets the property ERROR_MSG with the Rhapsody generated errors to be stored in the ERROR_MSG column in the ELR_ERROR table.

Sets the property [REDACTED] ENVIRONMENT with the Rhapsody Variable ECLIR_[REDACTED] ENVIRONMENT to be stored in the [REDACTED] ENVIRONMENT column in the ELR_MESSAGE table.

Sets the content of the message back to the original to be stored in the ORIGINAL_MESSAGE column in the ELR_MESSAGE table if needed by loading the SnapShot if the id is not null.

Javascript

```
//
=====
// _____ INITIAL VARIABLES (start
)
//
=====

var next = output.append(input[0]);

//
=====
// _____ INITIAL VARIABLES (end
)
//
=====

//
=====
// _____ SET MESSAGE PROPERTIES FOR DB INSERT (start
)
//
=====

next.setProperty('VALIDATION_TYPE', 'Rhapsody');
next.setProperty('ERROR_CATEGORY', 'FATAL');
next.setProperty('ERROR_TYPE', 'Rhapsody');
next.setProperty('PROCESSING_STEP', 'Error Inserts to DB: ' + next.getProperty('rhapsody:Error
Route') + ' (Route) ->' + next.getProperty('rhapsody:ErrorFilterName' + ' (Filter)'));
//next.setProperty('ERROR_MSG', 'Internal Rhapsody Route Error. Message processing could not
continue: ' + next.getErrors());
var ERROR_MSG = 'Internal Rhapsody Route Error. Message processing could not continue: ' + next
.getErrors();
next.setProperty('ERROR_MSG', ERROR_MSG);

//Send the message to the Error Queue if it was caused by a database connectivity issue.
if (ERROR_MSG.indexOf('The connection is broken and recovery is not possible.') != -1 || ERROR
_MSG == 'Internal Rhapsody Route Error. Message processing could not continue: ') {
    next.setProperty('SendToErrorQueue', true);
}
```

Javascript

```
//This is a Rhapsody variable that should be set up during configuration.
var █████_ENVIRONMENT = getVariable("ECLIR_█████_ENVIRONMENT");
if (█████_ENVIRONMENT == null || █████_ENVIRONMENT == "") {
    █████_ENVIRONMENT = "VARIABLE NOT SETUP IN RHAPSODY";
}
next.setProperty('█████_ENVIRONMENT', █████_ENVIRONMENT);

// Load the snapshot back into the message.
if (next.getProperty("SnapshotIDofOrigMsg") != null) {
    next.text = next.loadMessageSnapshotForId(next.getProperty("SnapshotIDofOrigMsg")).text;
}

//
=====
// _____ SET MESSAGE PROPERTIES FOR DB INSERT (end
// _____
//
=====
```

Destinations

Name	Page
Output Condition	
SendToErrorQueue = 'true' ignoring case	
! Error Queue..... Error Queue	168
Error Connector	Page
db ELR Errors Route Errors..... Database LookUp	166



js Prepare for DB Insert

JavaScript (v.1)

On ECLIR - ELR Errors route

Filter

Notes

Purpose: This JavaScript filter

Checks if the XML output is an empty message. If it is not, the errors are sent to the *db Insert ELR Errors* Database LookUp to be recorded in the ECLIR database table ELR_ERROR.

Sets the property █████_ENVIRONMENT with the Rhapsody Variable ECLIR_█████_ENVIRONMENT to be stored in the █████_ENVIRONMENT column in the ELR_MESSAGE table.

Javascript

```
//
=====
// _____ INITIAL VARIABLES (start
// _____
```

Javascript

```
//
=====

var next = output.append(input[0]);

//
=====
//_____INITIAL VARIABLES (end
)_____
//
=====

//
=====
//_____SET MESSAGE PROPERTIES FOR DB INSERT (start
)_____
//
=====

//If there were no errors to be written to the database, send to ECLIR - ELR Sink
if (next.text == '') {
    next.setProperty('ErrorsPresent', false);
}
else { next.setProperty('ErrorsPresent', true); }

//This is a Rhapsody variable that should be set up during configuration.
var _____ENVIRONMENT = getVariable("ECLIR _____ENVIRONMENT");
if (_____ENVIRONMENT == null || _____ENVIRONMENT == "") {
    _____ENVIRONMENT = "VARIABLE NOT SETUP IN RHAPSODY";
}
next.setProperty('_____ENVIRONMENT', _____ENVIRONMENT);

//
=====
//_____SET MESSAGE PROPERTIES FOR DB INSERT (end
)_____
//
=====
```

Destinations

Name	Page
Output Condition: Errors Present	
ErrorsPresent = 'true' ignoring case	
 db Insert ELR Errors.....Database LookUp	159
Error Connector	Page
 ECLIR - ELR Sink.....Sink	14

**no-op to Sink**

Filter

No-operation

On ECLIR - ELR Errors route

Notes

Purpose: This No-operation filter sends all messages to the *ECLIR - ELR Sink* after queries are completed or if empty result sets are received.

Destinations

Name	Page
 ECLIR - ELR Sink.....Sink	14

**db ELR Errors Route Errors**

Filter

Database LookUp

On ECLIR - ELR Errors route

Notes

Purpose: This Database Lookup filter inserts an error record into the ELR_ERROR table in the ECLIR database in the event an unhandled error is caught by the *ECLIR - ELR Errors Error Handler*.

Configuration Properties

Maximum Concurrency	1	
Database	[REDACTED]	
Host	[REDACTED]	
Port	[REDACTED]	
Database Name	[REDACTED]	
Username	[REDACTED]	
Password	[REDACTED]	
Additional Connection Properties	Property Name	Property Value
	[REDACTED]	[REDACTED]
Configuration File	[REDACTED]	
Message Type	Plain text	
Definition		
Query Timeout		
Deadlock Retry Attempts	5	
Deadlock Retry Delay	100	
Socket Timeout	0	

Configuration File: Database LookUp 2.xml

```

<statement name="ELR_MESSAGE ID Does Not exist" ide:type="executeSql" test="Null($ELR_MESSAGE_ID)" ide:summary="The ELR_MESSAGE ID does not exist in the ELR_MESSAGE table. Create an entry in that table to create an ID." ide:allowCompoundStatements="true">
  <sql>INSERT INTO ELR_MESSAGE (
ORIGINAL_MESSAGE
, █████ ENVIRONMENT
,SUCCESS_FL
,INSERT_DATE
)
VALUES (
$messageContent
, █████ ENVIRONMENT
,0
,SYSDATETIME()
)</sql>
  <statement ide:type="compoundSqlStatement">
    <sql>SELECT \@ \@IDENTITY AS ELR_MESSAGE_ID</sql>
    <substituteColumn name="ELR_MESSAGE_ID">$ELR_MESSAGE_ID</substituteColumn>
  </statement>
  <statement name="Insert into ELR_ERROR table" ide:type="executeSql" test="NotNull(@ELR_MESSAGE_ID)" ide:summary="Add error record to the ELR_ERROR table." ide:allowCompoundStatements="true">
    <sql>INSERT INTO ELR_ERROR (
ELR_MESSAGE_ID
,VALIDATION_TYPE
,ERROR_CATEGORY
,ERROR_TYPE
,PROCESSING_STEP
,ERROR_MSG
,INSERT_DATE
)
VALUES (
@ELR_MESSAGE_ID
,$VALIDATION_TYPE
,$ERROR_CATEGORY
,$ERROR_TYPE
,$PROCESSING_STEP
,$ERROR_MSG
,SYSDATETIME()
)</sql>
  </statement>
</statement>
  <statement name="Insert into ELR_ERROR table" ide:type="executeSql" test="NotNull($ELR_MESSAGE_ID)" ide:summary="Add error record to the ELR_ERROR table." ide:allowCompoundStatements="true">
    <sql>INSERT INTO ELR_ERROR (
ELR_MESSAGE_ID
,VALIDATION_TYPE
,ERROR_CATEGORY
,ERROR_TYPE
,PROCESSING_STEP
,ERROR_MSG
,INSERT_DATE
)

```

Configuration File: Database LookUp 2.xml

```

VALUES (
  $ELR_MESSAGE_ID
  , $VALIDATION_TYPE
  , $ERROR_CATEGORY
  , $ERROR_TYPE
  , $PROCESSING_STEP
  , $ERROR_MSG
  , SYSDATETIME()
) </sql>
</statement>
</config>

```

Destinations

Name	Page
 no-op REH to Sink..... No-operation	162

**Error Queue****Error Queue**

On ECLIR - ELR Errors route

Filter

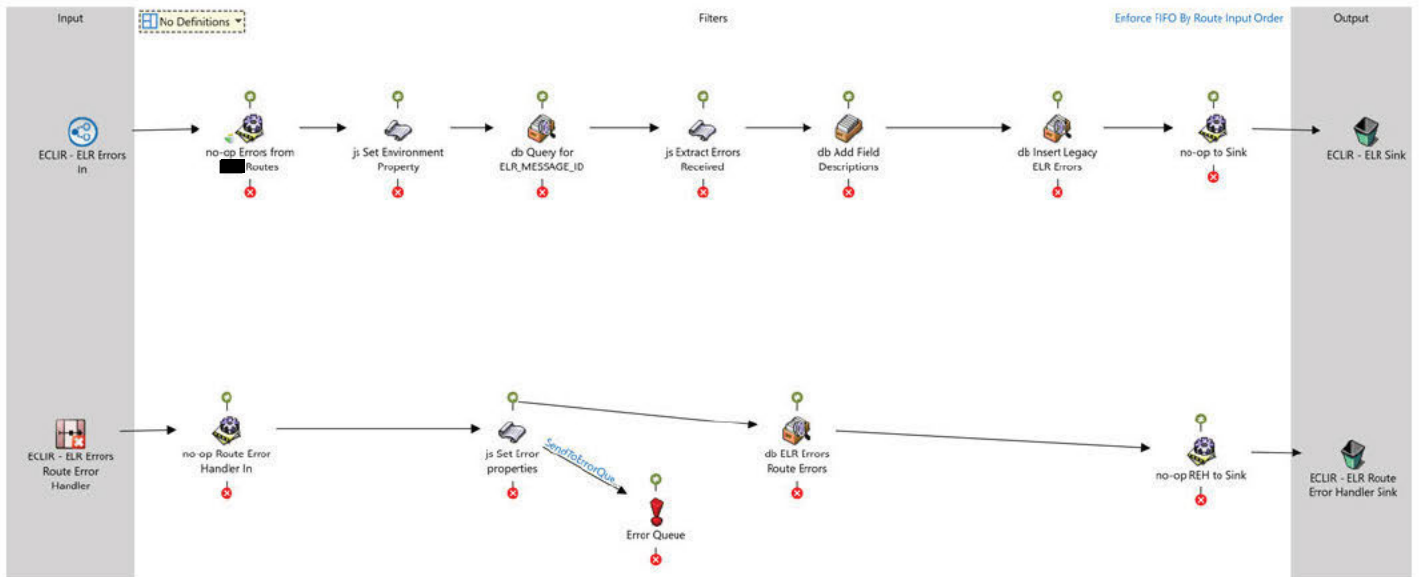
Configuration Properties

Error Message	Sent from error filter
---------------	------------------------



ECLIR - ELR Legacy Route Errors

Route



Notes

Purpose: This route receives errors from legacy routes to be recorded in the ELR_ERROR table in ECLIR.

Inputs

Inputs	Page
ECLIR - ELR Errors In..... Dynamic Router	6

Outputs

Outputs	Page
ECLIR - ELR Sink..... Sink	14
ECLIR - ELR Route Error Handler Sink..... Sink	13

Filters

Name	Page
db Add Field Descriptions..... Database Message Extraction	176
db ELR Errors Route Errors..... Database LookUp	180
db Insert Legacy ELR Errors..... Database LookUp	182
db Query for ELR_MESSAGE_ID..... Database LookUp	177
Error Queue..... Error Queue	176
js Extract Errors Received..... JavaScript (v.1)	171
js Set Environment Property..... JavaScript (v.1)	179
js Set Error properties..... JavaScript (v.1)	174
no-op Errors from TDH Routes..... No-operation	173
no-op REH to Sink..... No-operation	174
no-op Route Error Handler In..... No-operation	173

Name	Page
 no-op to Sink..... No-operation	179

Details

General	
Priority	Normal
Startup State	Startup Third
FIFO Mode	Enforce FIFO By Route Input Order
Stop route on error message	False



js Extract Errors Received

Filter

JavaScript (v.1)

On ECLIR - ELR Legacy Route Errors route

Notes

Purpose: This JavaScript filter outputs an XML file that follows the ECLIR - Error.xsd format for use to write these errors to the ECLIR database in the *ECLIR - ELR Errors* route. These are errors received from the legacy XXXXXXXXXX ELR routes:

- 1 - ELR
- 3 - Spreadsheet
- 4 - HL7
- 5 - Msg Splitters

Javascript

```
//
=====
// _____ INITIAL VARIABLES (start
)
//
=====

var next = output.append(input[0]);
var ELR_MESSAGE_ID = next.getProperty('ELR_MESSAGE_ID');

//
=====
// _____ INITIAL VARIABLES (end
)
//
=====

//
=====
// _____ BUILD ERROR XML (start
)
//
=====

if (ELR_MESSAGE_ID != null) {

    next.text = '<message></message>';

    next.setField('/message/errorRecord/ELR_MESSAGE_ID', ELR_MESSAGE_ID);
    next.setField('/message/errorRecord/ERROR_TYPE_DETAIL', em_ErrType);
    var em_ErrType = next.getProperty('em_ErrType');
    var em_BodyText = next.getProperty('em_BodyText');
    em_BodyText = em_BodyText.substr(em_BodyText.indexOf(',') + 5);
    em_BodyText = em_BodyText.replace(/\x0d\x0a/g, '');
    em_BodyText = em_BodyText.replace(/\r\n/g, '');
```

Javascript

```

if (em_ErrType != null) {

switch(em_ErrType) {
case 'HL7-ReqSegment':
    next.setField('/message/errorRecord/ERROR_CATEGORY', 'FATAL');
    next.setField('/message/errorRecord/PROCESSING_STEP', 'From Legacy Route: 1 - ELR');
    next.setField('/message/errorRecord/FIELD', next.getProperty('missingRequired'));
    next.setField('/message/errorRecord/ERROR_MSG', em_BodyText);
    break;
case 'HL7-DupeMsg':
    next.setField('/message/errorRecord/ERROR_CATEGORY', 'FATAL');
    next.setField('/message/errorRecord/PROCESSING_STEP', 'From Legacy Route: 1 - ELR');
    next.setField('/message/errorRecord/FIELD', 'MSH-3|MSH-4|MSH-10');
    next.setField('/message/errorRecord/FIELD_VALUE', next.getProperty('UniqueMsgKey'));
    next.setField('/message/errorRecord/ERROR_MSG', em_BodyText);
    break;
case 'SPR-TestNotPerf':
    next.setField('/message/errorRecord/ERROR_CATEGORY', 'FATAL');
    next.setField('/message/errorRecord/PROCESSING_STEP', 'From Legacy Route: 3 - Spreads
heet');
    next.setField('/message/errorRecord/ERROR_MSG', em_BodyText);
    break;
case 'HL7-LOINC':
    next.setField('/message/errorRecord/ERROR_CATEGORY', 'FATAL');
    if (next.getProperty('loincErr') != null) {
        if (next.getProperty('loincErr').indexOf('OBR-4') != -1) {
            next.setField('/message/errorRecord/FIELD', 'OBR-4');
        }
        else if (next.getProperty('loincErr').indexOf('OBX-3') != -1) {
            next.setField('/message/errorRecord/FIELD', 'OBX-3');
        }
    }
    next.setField('/message/errorRecord/PROCESSING_STEP', 'From Legacy Route: 4 - HL7');
    next.setField('/message/errorRecord/ERROR_MSG', em_BodyText);
    break;
case 'Info-LOINC':
    next.setField('/message/errorRecord/ERROR_CATEGORY', 'FATAL');
    next.setField('/message/errorRecord/PROCESSING_STEP', 'From Legacy Route: 5 - Msg Splitters
');
    next.setField('/message/errorRecord/FIELD', 'OBX-3.1');
    next.setField('/message/errorRecord/FIELD_VALUE', next.getProperty('missingLoinc'));
    next.setField('/message/errorRecord/ERROR_MSG', em_BodyText);
    break;
default:
    next.text = input[0];
    break;
}
}

else if (next.getProperty('nhbs_SendingFac').toUpperCase() == '██████████ CPHL' && next.getProp
erty('nhbs_PatientLName').toUpperCase() == 'NHBS') {
    next.setField('/message/errorRecord/ERROR_CATEGORY', 'FATAL');
    next.setField('/message/errorRecord/PROCESSING_STEP', 'From Legacy Route: 4 - HL7');
    next.setField('/message/errorRecord/ERROR_MSG', 'Message sent to sink for TASK0590582 Nat
ional HIV Behavioral Surveillance');
}

else if (next.getProperty('obx_obsId') == '29553-5') {
    next.setField('/message/errorRecord/ERROR_CATEGORY', 'FATAL');
}

```

Javascript

```

next.setField('/message/errorRecord/PROCESSING_STEP', 'From Legacy Route: 5 - Msg Splitters
');
next.setField('/message/errorRecord/ERROR_MSG', 'Message sent to sink because it is an AOE
message. ');
}
} // (END) if (ELR_MESSAGE_ID != null) (END)

//
=====
// _____ BUILD ERROR XML (end
)
//
=====

```

Destinations

Name	Page
 db Add Field Descriptions.....Database Message Extraction	176

**no-op Errors from [REDACTED] Routes**

Filter

No-operation

On ECLIR - ELR Legacy Route Errors route

Notes

Purpose: This No-operation filter receives ELR messages that have erred from the existing [REDACTED] routes to be recorded in the ECLIR database. There is a timed hold of 5 minutes to prevent "race" conditions for messages to insure a unique message key ID has been created in the ELR_MESSAGE table for that message in the ECLIR database.

Message Collector: Timed Hold

Message Set Timeout	5
Timeout Units	Minutes
Timeout Destination	Process as Usual

Destinations

Name	Page
 js Set Environment Property.....JavaScript (v.1)	179

**no-op Route Error Handler In**

Filter

No-operation

On ECLIR - ELR Legacy Route Errors route

Notes

Purpose: This No-operation filter receives messages from the *ECLIR - ELR Errors Route Error Handler* when an unhandled error occurs anywhere in the route.

Destinations

Name	Page
js Set Error properties..... JavaScript (v.1)	174



no-op REH to Sink

Filter

No-operation

On ECLIR - ELR Legacy Route Errors route

Notes

Purpose: This No-operation filter sends messages from the *ECLIR - ELR Errors Route Error Handler* to the *ECLIR - ELR Route Error Handler Sink* after the errors have been stored into the ELR_ERROR table in the ECLIR database.

Destinations

Name	Page
ECLIR - ELR Route Error Handler Sink..... Sink	13



js Set Error properties

Filter

JavaScript (v.1)

On ECLIR - ELR Legacy Route Errors route

Notes

Purpose: This JavaScript filter

Sets the property VALIDATION_TYPE with the type of validation error to be stored in the VALIDATION_TYPE column in the ELR_ERROR table. The type is set to 'Rhapsody' because the error is an internal Rhapsody error.

Sets the property ERROR_CATEGORY with the category of the error to be stored in the ERROR_CATEGORY column in the ELR_ERROR table. The category is set to 'FATAL' because the message stops processing.

Sets the property ERROR_TYPE with the type of error to be stored in the ERROR_TYPE column in the ELR_ERROR table. The type is set to 'Rhapsody' because the error is an internal Rhapsody error.

Sets the property PROCESSING_STEP with the Rhapsody route name and filter name to be stored in the PROCESSING_STEP column in the ELR_ERROR table.